

Eventos de domínio do FestaLink

Este documento descreve quais eventos o backend publica, por qual caminho eles trafegam no RabbitMQ, qual o formato do payload e quem é o consumidor responsável. Os eventos materializam a comunicação assíncrona entre a parte síncrona da API (REST) e a parte que notifica o prestador em tempo real.

A topologia no broker

Todo evento entra num único exchange durável no RabbitMQ. O nome vem da variável `EVENTOS_EXCHANGE` no `.env`, com padrão `festalink.eventos`. O tipo do exchange é `topic`. Escolhi `topic` em vez de `direct` porque facilita adicionar novos tipos de evento depois sem mexer no binding: basta seguir o padrão `reserva.` (ou `cliente.`, se um dia precisar) e a fila do worker pega automaticamente.

A fila do consumidor (`festalink.worker` por padrão, controlada por `EVENTOS_FILA_WORKER`) é declarada como `durable: true` e está bindada ao exchange com a routing key `reserva.#`, que é configurável em `EVENTOS_ROUTING_KEYS` (lista separada por vírgula). Cada mensagem é publicada com `persistent: true`, então o broker grava em disco antes de confirmar o `publish`. Combinado com o `confirmChannel` do producer, isso me dá a garantia de que toda mensagem que a API considera "publicada com sucesso" já está no disco do RabbitMQ.

A routing key de cada evento é derivada do `type` do envelope por uma função que converte PascalCase pra dot-snake. Então `ReservaSolicitada` vira `reserva.solicitada`, `ReservaAprovada` vira `reserva.aprovada`, e por aí vai.

O envelope

Todo evento publicado segue o mesmo formato:

```
{
  "type": "NomeDoEvento",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": { ... }
}
```

O `type` e o `timestamp` são fixos em todos os eventos. O `payload` muda conforme o evento e está documentado abaixo, evento a evento.

A lista de eventos

Evento	Use case que publica	Quando dispara	Routing key	Fila do consumidor
ReservaSolicitada	CriarReserva	Depois que a reserva é persistida com status pendente	reserva.solicitada	festalink.worker
ReservaAprovada	AprovarReserva	Depois que a reserva muda para aprovada	reserva.aprovada	festalink.worker
ReservaRecusada	RecusarReserva	Depois que a reserva muda para recusada	reserva.recusada	festalink.worker

A publicação acontece sempre depois do commit da transação, e roda em microtarefa (`Promise.resolve().then(...)`) pra não bloquear o caminho da resposta HTTP. Se a publicação falhar por algum motivo, o erro é logado mas não derruba a requisição que originou o evento. Isso é deliberado: a reserva já está no banco, não faz sentido devolver erro 500 pro cliente só porque o broker piscou.

Os payloads

ReservaSolicitada

```
{
  "type": "ReservaSolicitada",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6,
    "dataEvento": "2026-06-05",
    "horaInicio": "19:00",
    "horaFim": "22:00"
  }
}
```

```
}  
}
```

Carrega data e horário porque o app do prestador exibe esses campos diretamente na notificação de pedido novo, sem precisar fazer outra chamada à API.

ReservaAprovada

```
{  
  "type": "ReservaAprovada",  
  "timestamp": "2026-05-25T23:41:47.236Z",  
  "payload": {  
    "reservaId": 4,  
    "salaoId": 4,  
    "clienteId": 6  
  }  
}
```

Payload enxuto. O app só precisa saber qual reserva mudou de status; os detalhes ele consulta sob demanda.

ReservaRecusada

```
{  
  "type": "ReservaRecusada",  
  "timestamp": "2026-05-25T23:41:50.111Z",  
  "payload": {  
    "reservaId": 4,  
    "salaoId": 4,  
    "clienteId": 6  
  }  
}
```

Mesma forma de `ReservaAprovada`. Trato como dois eventos distintos em vez de um único `ReservaStatusMudou` porque acho mais legível no consumidor e porque o roteamento por routing key fica mais natural.

O caminho de uma mensagem

Passo a passo do que acontece quando uma reserva é criada e o prestador conectado por WebSocket precisa ser notificado:

1. O cliente manda `POST /reservas` na API. O `CriarReserva` valida, abre transação, faz o `INSERT` no SQLite com status `pendente` e dá `commit`.
2. Logo depois do commit, ainda dentro do mesmo use case, é disparada uma microtarefa que chama `eventPublisher.publicar(reservaSolicitada(reserva))`.
3. O `RabbitMQEventPublisher` serializa o envelope, deriva a routing key (`reserva.solicitada`), e chama `publish` no exchange `festalink.eventos` com `persistent: true`. Como o channel está em modo `confirm`, a Promise do `publish` só resolve quando o broker confirma a recepção.
4. O RabbitMQ recebe a mensagem, identifica a fila `festalink.worker` (que está bindada com `reserva.#`), e enfileira ali.
5. O worker, conectado em modo consumer com `prefetch(1)`, recebe a mensagem. O callback faz `JSON.parse` no payload e chama `wsNotifier.notificarPrestadorDoSalao(salaoId, evento)`.
6. O `WebSocketNotifier` consulta o `SalaoRepository` pra descobrir o `prestadorId` daquele salão, procura no Map de conexões abertas, e se houver socket aberto pra esse prestador, faz `send` no envelope.
7. Se a entrega der certo (sem exceção), o consumer faz `ack` manual da mensagem e o RabbitMQ remove ela da fila. Se der erro de processamento, faz `nack` com requeue e a mensagem volta pra fila pra ser reentregue.

A propriedade central dessa topologia é a durabilidade: se o worker estiver offline na hora que a API publicar, a mensagem fica enfileirada em `festalink.worker` e é entregue assim que o worker reconectar. A API segue funcionando sem se importar com o estado do worker.

O WebSocket de notificação

O `WebSocketNotifier` mora dentro do processo do worker e abre um servidor WebSocket na porta `WS_PORT` (padrão 3001). A autenticação acontece no handshake: o cliente conecta passando o JWT na query string (`?token=<jwt>`). O notificador valida o token usando o mesmo `TokenService` que o middleware HTTP usa pra REST, exige que o `papel` do payload seja `prestador`, e indexa o socket pelo `prestadorId`. Se o token estiver ausente, expirado, inválido ou o papel for `cliente`, a conexão é fechada com um código de erro apropriado.

Logo depois do handshake, o cliente recebe `{"type": "Conectado", "prestadorId": N}` pra confirmar a sessão. A partir daí, mensagens publicadas pela API e entregues ao worker chegam ao prestador no mesmo formato do envelope mostrado acima.